

Contents

1	Implementation Plan: Unified bi-material support — VOLUME contrast + FAULT contrast (TPV31 + TPV6/7)	1
1.1	Overview	1
1.2	What already exists (verified this session — do NOT re-derive)	2
1.3	What blocks bi-material FAULT today	2
1.4	Constraints (apply to every phase)	2
1.5	Key math	2
2	PART A — Bi-material VOLUME: central-flux contrast guard	3
2.1	Phase A1: contrast predicate + tolerance member	3
2.2	Phase A2: apply the guard in the bi-material central build + diagnostics	4
2.3	Phase A3: config + CLI wiring (volume guard)	4
2.4	Phase A4: volume regression + TPV31 Frontera A/B	5
3	PART B — Bi-material FAULT: per-side fault Riemann	5
3.1	Phase B0: VERIFY the existing per-side fault Riemann (INVESTIGATION — Rule 5, do before B1-B3)	5
3.2	Phase B1: per-side material assignment in the fault DOF setup	6
3.3	Phase B2: relax the homogeneity guards for the matrix path	8
3.4	Phase B3: across-fault material schema	8
4	PART C — Problem setups	9
4.1	Phase C1: TPV31 (bi-material VOLUME) — contrast-guard fix jobs ONLY	9
4.2	Phase C2: TPV6 / TPV7 (bi-material FAULT) — NEW problem	10
5	PART D — Testing Strategy	12
6	Risk Assessment	12

1 Implementation Plan: Unified bi-material support — VOLUME contrast + FAULT contrast (TPV31 + TPV6/7)

Worktree: `.claude/worktrees/fix-mixedflux-hetero` (branch `worktree-fix-mixedflux-hetero`) Date: 2026-06-06 All math is ASCII (no LaTeX) per project convention.

Supersedes / folds in: - `document/mixed_flux_dev/PLAN_bimaterial_central_contrast_guard_2026-06-06.md` (the VOLUME contrast guard) + its review `REVIEW_plan_bimaterial_contrast_guard_2026-06-06.md` (fixes R-001..R-007 are incorporated below). Companion docs: - `debug_document/tpv31_debug_document/drdg3d_central_flux_comp`
- TPV6/7 spec: `tpv6/benchmark_document/2007RuthRalphletter2.pdf` (Harris/Day letter, Jan 2007), summarized in Part C.

1.1 Overview

Generalize the SEAS dynamic-rupture stack so a single run can carry BOTH kinds of material heterogeneity at once: - **Bi-material VOLUME** — material varies between interior elements (TPV31 depth layers; SAFS CVM). Handled by `BimaterialWaveOperator`; this plan adds the central-flux **contrast guard** so the non-dissipative mixed flux is not applied across an impedance jump inside the fault-adjacent corridor. - **Bi-material FAULT** — material differs across the fault plane itself (TPV6/7; SAFS fault separating different rock). The fault Riemann math is ALREADY per-side; this plan wires the per-side materials into the fault DOF setup, relaxes the homogeneity guards, verifies the per-side imposed-state flux, and adds an across-fault material schema.

The unifying design: BOTH volume and fault heterogeneity are read from the SAME per-element material pool (`BimaterialWaveOperator::FluxForElem_`). A fault DOF's two impedances come from its +/- adjacent elements;

an interior face's contrast comes from its two elements. TPV31 (same material across the fault) reduces to the current homogeneous-fault path byte-exactly; TPV6/7 (uniform per side) exercises the per-side fault Riemann with a no-op volume guard; SAFS exercises both.

1.2 What already exists (verified this session — do NOT re-derive)

- `FaultFaceFlux::ComputeTrialTraction` (`fault_face_flux.cpp:52-83`), `BuildImposedState` (:271-308), the friction solve (:201-266), `EvaluateLSW`, `Evaluate`, `EvaluateADER` — all consume per-side `Zp_plus/Zp_minus`, `Zs_plus/Zs_minus`, and harmonic-mean `eta_p/eta_s` from `DOFData`. The math is per-side-ready.
- The interior fault-flux $\rightarrow$ bulk conversion applies PER-SIDE A: `FluxForElem_(elem_plus).Interior(can_n, Q_imp_plus_g, ...)` and `FluxForElem_(elem_minus).Interior(...)` (`wave_operator.inl:3048-3051`). For a `BimaterialWaveOperator`, `FluxForElem_` returns the per-element Jacobian.
- `BimaterialWaveOperator` builds a per-element `GodunovFlux` pool from the material field and per-face bi-material upwind matrices (`bimaterial_wave_operator.inl`).
- The fault Riemann is single-valued in traction (conservative) by construction.

1.3 What blocks bi-material FAULT today

1. Setup hard-codes equal materials: `dynamic/spatial_setup.hpp:85,131` `d.Zp_plus = d.Zp_minus = rho*cp`; `d.Zs_plus = d.Zs_minus = rho*cs`; (single local material; never reads the two fault sides).
2. Homogeneity guards ABORT when `Zp_plus != Zp_minus`: `fault_face_flux.cpp:359, 499, 778, 915, 1033` (across `Evaluate` / `EvaluateLSW` / `EvaluateADER` variants).
3. Scalar `WaveOperator::FluxForElem_` returns the single `flux_` (`wave_operator.hpp:966`) $\rightarrow$ bi-material fault REQUIRES the matrix (`BimaterialWaveOperator`) path.
4. No across-fault material kind in the spatial schema (kinds: `constant`, `depth_profile_1d` [depth-only, symmetric across the fault], `cvmh` sidecar).
5. No TPV6/7 config, mesh, or stations.

1.4 Constraints (apply to every phase)

- BYTE-EXACT regression contract (`CLAUDE.md`): every new behavior defaults OFF/inert. Same material across the fault (TPV31, all homogeneous TPV/BP5) $\rightarrow$ `Zp_plus == Zp_minus` $\rightarrow$ per-side path reduces to today's flux bit-for-bit. Volume guard `contrast_tol < 0` $\rightarrow$ disabled.
- DO NOT change the fault FRICTION law, the imposed-state EQUATIONS, or the scalar `WaveOperator` path. The fault generalization is wiring + guard relaxation, NOT a new Riemann derivation (the per-side equations already exist; Part B VERIFIES them).
- Bi-material FAULT requires the matrix operator: a `Zp_plus != Zp_minus` DOF on the scalar `WaveOperator` path MUST still abort (the scalar `FluxForElem_` cannot apply per-side A). Relax the guard ONLY for the matrix path.
- MPI: per-side material for a SHARED fault face needs the cross-rank neighbour material (`shared_face_neighbour_material` / `ExchangeBiMaterialNeighbours`); both ranks must agree on the two impedances and the +/- side assignment (`shared_fault_elem1_on_plus_`, `wave_operator.inl:464-540`).
- NO local full-mesh runs (memory `feedback-no-local-mesh-runs`): local acceptance = compile + unit/parallel tests + the serial planar harness; production = Frontera, user-submitted.
- Gmsh meshes MUST be v2.2 (`-format msh22`) — `miniapps/seas/CLAUDE.md`.
- Build in the worktree: `make MFEM_DIR=../.. MFEM_BUILD_DIR=$MAIN MFEM_INC_DIR=$MAIN MFEM_LIB_DIR=$MAIN <target> (MAIN=/Users/chunhuizhao/projects/seas-mfem-spatial-dyn-driver)`.
- Do NOT revert a prior fix without citing the debug doc + approval.

1.5 Key math

1.5.1 Bi-material fault Riemann (already in the code; Part B verifies it)

Fault-local frame: n =normal, t_1 =dip, t_2 =strike (`DOFData`: $SXX=\sigma_n$, $SXY=\tau_{u1}$, $SXZ=\tau_{u2}$; $VX/VY/VZ$ = normal/dip/strike velocity). Per-side impedances $Zp_s = \rho_s \cdot cp_s$, $Zs_s = \rho_s \cdot cs_s$ for s in $\{+, -\}$; jumps $[[a]] = a^- - a^+$.

```

eta_p = Zp^+ Zp^- / (Zp^+ + Zp^-)      eta_s = Zs^+ Zs^- / (Zs^+ + Zs^-)
sigma_n_trial = eta_p ( [[v_n]] + sigma_n^+/Zp^+ + sigma_n^-/Zp^- )
tau1_trial    = eta_s ( [[v_dip]] + tau1^+/Zs^+ + tau1^-/Zs^- )
tau2_trial    = eta_s ( [[v_str]] + tau2^+/Zs^+ + tau2^-/Zs^- )
friction (LSW): strength = mu(slip)*max(0,sigma_n) + C0;
  if |tau_trial| > strength: V = (|tau_trial| - strength)/eta_s,
      tau_corr = tau_trial * strength/|tau_trial|;
  else V=0, tau_corr = tau_trial.
imposed velocity (per side): v^+{-,imp} = v^- - (1/Z^-)(t_corr - t^-),
      v^+{+,imp} = v^+ + (1/Z^+)(t_corr - t^+); t^+{imp}=t_corr
flux per side: F^s = A_s . (T . Q^s{imp}) (A_s = that side's Jacobian; the
  call site applies A_plus to + and A_minus to - via FluxForElem_).

```

Homogeneous reduction: $Zp^+ = Zp^- \Rightarrow eta_p = Zp/2$ etc., reproducing the current flux.

1.5.2 Volume contrast (central-flux guard)

Interior - Central = 0.5 |A| (Q_self - Q_nbr) (the SPD dissipation central discards). Across an impedance contrast central is non-dissipative AND not impedance-weighted, so it seeds the leak. Strong-contrast predicate (threshold on impedance): $cZp = |Zp1-Zp2|/\max(Zp1,Zp2)$; $cZs = |Zs1-Zs2|/\max(Zs1,Zs2)$ strong := tol >= 0 && $\max(cZp,cZs) > \text{tol}$ (default tol = 0.05; see prior plan)

2 PART A — Bi-material VOLUME: central-flux contrast guard

(Folds PLAN_bimaterial_central_contrast_guard_2026-06-06.md with review fixes R-001/R-002/R-003/R-005/R-006/R-007 applied.)

2.1 Phase A1: contrast predicate + tolerance member

2.1.1 Goal

A pure, tested impedance-contrast predicate and a WaveOperator tolerance member (default -1 = disabled), with zero behavior change.

2.1.2 Files to Modify

- dynamic/godunov_flux_bimaterial.hpp/.cpp — add static bool BimaterialFlux::IsStrongContrast(const GodunovFlux& a, const GodunovFlux& b, real_t tol) returning tol >= 0 && std::max(rel(a.GetZp()),b.GetZp rel(a.GetZs()),b.GetZs())) > tol, with rel(x,y)=|x-y|/std::max(x,y); return false if either max impedance <= 0.
- dynamic/wave_operator.hpp — protected real_t mixed_flux_contrast_tol_ = -1.0; — public void SetMixedFluxContrastTol(real_t) / real_t GetMixedFluxContrastTol() const.

2.1.3 Detailed Requirements

1. IsStrongContrast symmetric, side-effect free; tol < 0 => false.
2. Threshold on $\max(cZp,cZs)$ deliberately (R-007): catches ANY impedance contrast a non-dissipative central flux would mishandle; do NOT reduce to Zp-only.
3. No call site passes a non-negative tol in A1 (tree byte-identical after A1).

2.1.4 Acceptance Criteria

- ☐ Compiles (serial Mesh + ParMesh TUs).
- ☐ seas_test_bimaterial_contrast_guard (NEW) Phase-A1: false for tol<0; false for equal materials; true for TPV31 5 km pair (cZs ~14.8%) at tol=0.05; false for the linear-gradient adjacent pair (cZs ~0.36%, h=50 m) at tol=0.05.
- ☐ git diff shows no numerics change (member unused).

2.1.5 Dependencies: depends on nothing; required by A2, A3.

2.2 Phase A2: apply the guard in the bi-material central build + diagnostics

2.2.1 Goal

With `tol >= 0`, fault-adjacent corridor faces whose two elements are a strong contrast are removed from `central_flux_face_set_` (LOCAL + SHARED), so they dispatch to the existing bi-material upwind; a rank-0 contrast histogram prints.

2.2.2 Files to Modify

- `dynamic/bimaterial_wave_operator.inl::BuildPerFaceCentralFluxMatrices_ (:515):`
 - LOCAL loop (:550–571): after `ResolveFaceFluxOperands_(mesh_face, flux_e1, flux_e2, nor)`, if `BimaterialFlux::IsStrongContrast(*flux_e1,*flux_e2, mixed_flux_contrast_tol_)` record `mesh_face` in `std::vector<int>` reclassified and continue (skip central matrices).
 - SHARED loop (:660–690): build the neighbour `GodunovFlux` from `shared_face_neighbour_material_.at(mesh_fa` and test the same predicate.
 - After both loops: assert the upwind fallback exists, then erase (R-002): `for (int f: reclassified){ MFEM_VERIFY(per_face_bimaterial_flux_[f][0][0].Height() == NUM_STATE, "contrast-guard: face " << f << " has no upwind fallback"); central_flux_face_set_.erase(f); }`
 - The existing invariant assert (:877) must still hold.
- Diagnostic: rank-0 contrast histogram over the PRE-filter corridor faces (bins `[0,1)/[1,5)/[5,10)/[10,20)/[>=20)%`) + reclassified N/M. R-005: count each shared face on the lower-rank owner only. R-006: the ENTIRE new block (filter + histogram + any MPI call) is under one `if (mixed_flux_contrast_tol_ >= 0)` evaluated identically on all ranks; for `tol<0` the function body is byte-identical to today (no new statements, no collectives); add a debug `MPI_Allreduce(min==max)` assert that `tol` is rank-uniform.

2.2.3 Detailed Requirements (delta vs prior plan)

1. R-003 BLOCKING step 0: read `ExchangeBiMaterialNeighbours_` end-to-end and document whether `shared_face_neighbour_material_` stores `{lambda,mu,rho}` or `{vp,vs,rho}`; build the neighbour `GodunovFlux` with the matching ctor (convert if needed: `mu=rho*vs^2, lambda=rho*vp^2-2mu`). Add a parallel assert that the neighbour `GodunovFlux.GetZp()/GetZs()` reproduce the peer's to 1e-10 rel (rank-consistent classification).
2. Guard is mode-agnostic (works for Adjacent AND AllContinuous); add an AllContinuous acceptance check (R-004).

2.2.4 Edge Cases

- Empty central set (`mixed_flux=none`) -> no-op, no histogram.
- Homogeneous bi-material -> 0 reclassified -> byte-exact.

2.2.5 Acceptance Criteria

- ☐ `tol=-1`: `central_flux_face_set_` identical to a reference build (byte-exact).
- ☐ `tol=0.05` on a contrast-corridor fixture (extend `test_bimaterial_mixed_flux_dispatch.cpp`): exactly the strong-contrast faces removed; uniform faces retained; AllContinuous case checked.
- ☐ `np=2` (extend `tests/parallel/test_bimaterial_mixed_flux_shared.cpp`): a strong shared face removed on BOTH ranks; uniform shared face retained; neighbour `GodunovFlux` impedance matches peer.
- ☐ `make test` green except the 3 known pre-existing failures.

2.2.6 Dependencies: depends on A1; required by A3, A4.

2.3 Phase A3: config + CLI wiring (volume guard)

- `spatial/code/spatial_friction.hpp/.cpp`: add `real_t numerics.mixed_flux_contrast_tol = -1.0`; parsed via `toml_real(n,"mixed_flux_contrast_tol",-1.0) (:1096 region)`.

- `drivers/spatial_dyn_driver.cpp`: CLI `--mixed-flux-contrast-tol` (parse near :576, CLI-over-TOML merge near :692); call `wave.SetMixedFluxContrastTol(...)` BEFORE `wave.SetMixedFluxMode(...)` (:1276); banner line near :884.
- Acceptance: `test_tpv_config_parse` extended; `--verify-dispatch` banner shows the value; default => “(disabled)” and byte-identical dispatch.

2.4 Phase A4: volume regression + TPV31 Frontera A/B

- `tests/unit/test_bimaterial_contrast_guard.cpp` hard-asserts (mechanism via the TWO-SIDED dissipation measure per R-001 below; face-set; byte-exact).
- `jobs/tpv31_spatial/mixedflux_rk45/..._contrastguard.sbatch` (NEW): gold job + `--mixed-flux-contrast-tol 0.05`.
- Findings doc records `sigma_n`-by-depth before/after.

2.4.1 R-001 corrected mechanism test (use everywhere a “central discards dissipation”

check is needed):

```
build upwind pair twice (self/nbr swapped) -> F_up_e1, F_up_e2; central -> F_ce.
for a pure normal-velocity jump Q across the 5 km contrast:
  D = (Q_e1 - Q_e2) . ( 0.5*(F_up_e1 + F_up_e2) - F_ce )
assert D > 0 across the contrast; D -> 0.5*Zp*jump^2 in the homogeneous limit (1e-9 rel).
```

3 PART B — Bi-material FAULT: per-side fault Riemann

3.1 Phase B0: VERIFY the existing per-side fault Riemann (INVESTIGATION — Rule 5, do before B1-B3)

3.1.1 Goal

Confirm, with evidence, that the existing per-side fault-flux math + all flux-conversion call sites are correct for `Zp_plus != Zp_minus`, so B2 can relax the guards safely.

3.1.2 Investigation tasks (read code; write findings to a doc)

1. Trace EVERY fault-flux -> bulk conversion and produce a TABLE (R-005) with one row per guarded Evaluate variant — Evaluate (`fault_face_flux.cpp`:359), EvaluateTotal (:499), EvaluateADER_LSW (:778), EvaluateLSW (:912), EvaluateADER_LSW_ForcedRupture (:1030) — and columns: {variant | its imposed-state->flux conversion site (file:line) | per-side A? Y/N | used by which driver path (spatial RK / spatial ADER / TPV* native / unused)}.
- Mult/RK interior path: `wave_operator.inl`:3048-3051 (verified: FluxForElem_ per side). Verified this session: `BimaterialWaveOperator` does NOT override Mult (no void Mult in `bimaterial_wave_operator.hpp`), so it inherits the base per-side conversion — record this in the table.
- ADER path: locate the fault-face conversion in the ADER corrector and confirm per-side A (or document it is RK-only for the spatial/TPV6 path).
- SHARED (cross-rank) fault path: confirm per-side A using the local element flux + the cross-rank neighbour material; confirm `shared_fault_elem1_on_plus_` orients +/- consistently on both ranks.
- Confirm the precomputed-face-flux path (`UsePrecomputedFaceFluxes`) does NOT bypass the live, friction-coupled fault conversion (fault flux cannot be precomputed).
2. Confirm the imposed-state equations (`BuildImposedState`) are the correct bi-material Riemann for unequal impedance (they are per-side). Verify in the LOCKED-fault limit only (set friction strength above `|tau_trial|` so `V=0` and `tau_corr=tau_trial`), where the fault Riemann reduces to the WELDED bi-material interface. Pin the field and sign (compression POSITIVE; `n` points to side 2): the stress reflection coefficient is $R_{\text{sigma}} = (Z2 - Z1)/(Z1 + Z2)$ and the velocity transmission is $T_v = 2 Z1/(Z1 + Z2)$ for a wave incident from side 1 (state which field each test compares — do NOT compare a velocity reflection against a stress

coefficient). The SLIPPING regime is NOT welded — validate it separately against SeisSol’s bi-material fault flux (Pelties 2014 / the v9.0.0 comparison doc), not against welded R/T.

3. Enumerate ALL homogeneity guards (`fault_face_flux.cpp:359,499,778,915,1033`) and which Evaluate variant each protects.

3.1.3 Deliverable

- `debug_document/tpv6_debug_document/bimaterial_fault_verification_2026-06-06.md` with: per-side-A confirmation for each call site; the analytic Riemann comparison result; the guard inventory; a VERDICT: “per-side fault flux is correct for the matrix path” or “defect found at ”.

3.1.4 Acceptance Criteria

- New unit test `seas_test_bimaterial_fault_riemann`: for a single fault face with `Zp_plus != Zp_minus` (TPV6 contrast), in the LOCKED limit ($V=0$), a normal-incidence P-wave from side 1 reproduces the welded star state: assert the interface normal STRESS reflection $R_{\text{sigma}} = (Z_2 - Z_1) / (Z_1 + Z_2)$ and normal VELOCITY transmission $T_v = 2 Z_1 / (Z_1 + Z_2)$ to $1e-9$, with the stated sign convention (compression positive, n to side 2). Uses `ComputeTrialTraction + BuildImposedState + per-side FluxForElem_-style` apply directly, no mesh.
- The conversion-site TABLE (task 1) is written and every matrix-path variant used by the spatial driver is marked per-side-A = YES (R-005).
- Verdict doc written; if a defect is found (R/T mismatch OR a variant whose conversion site is NOT per-side A), B2 is BLOCKED until a follow-up phase fixes the Riemann / that conversion site (do not relax the guard over a known defect).

3.1.5 Dependencies: depends on nothing; required by B1, B2.

3.2 Phase B1: per-side material assignment in the fault DOF setup

3.2.1 Goal

The fault DOF setup populates `Zp_plus/Zp_minus`, `Zs_plus/Zs_minus`, `eta_p/eta_s` from the ACTUAL +/- adjacent element materials (not a single local material), for local and shared fault faces.

3.2.2 Files to Modify

- `dynamic/spatial_setup.hpp` (`InitializeFaultDOFs_Spatial`, the :85 and :131 blocks): replace `d.Zp_plus = d.Zp_minus = rho*cp` with a per-side read computed by the **EPS-OFFSET RULE** (R-001) — NOT element centroids and NOT exactly at the fault face:
 - Let $\mathbf{x}_f$ be the fault-QP physical position and $\mathbf{n}$ the fault unit normal pointing to the +/-near side. Conceptually probe just inside each element at $\mathbf{x}_f \pm \text{eps} * \mathbf{n}$.
 - (R-102) CONSTRUCT the probe reference IP WITHOUT a physical->reference Newton solve: the fault QP already HAS a known element-reference IP in each adjacent element (`ip_face_p` in `elem_plus`, used to read `Q_plus`; likewise `ip_face_m`). Perturb it along the inward reference normal using the inverse Jacobian at the face QP: `ip_p = ip_face_p + eps_ref * normalize(J_plus^{-1} * n_inward_plus)` `ip_m = ip_face_m + eps_ref * normalize(J_minus^{-1} * n_inward_minus)` where `n_inward_*` points from the fault into that element and `eps_ref` is small enough that `ip_*` stays strictly inside the reference simplex (CLAMP and MFEM_VERIFY all barycentric coords in $[0,1]$). This uses only J^{-1} at the face QP — no `TransformBack`, no out-of-element / non-convergence risk. (If `TransformBack` is used as a fallback, MFEM_VERIFY it converged AND the result is in-element; abort loudly otherwise.)
 - Then `material.EvalAt(elem_plus, T_plus, ip_p, lam,mu,rho) -> Zp_plus,Zs_plus` and `material.EvalAt(elem_minus, T_minus, ip_m, ...) -> Zp_minus,Zs_minus`.
 - RATIONALE (do not “simplify” to centroids or to $\mathbf{x}_f$): a depth-only material (TPV31) gives the SAME value at `pt_p` and `pt_m` (they differ only in y , not z) -> `Zp_plus == Zp_minus` byte-exact, so the fault stays homogeneous on an asymmetric mesh. Element CENTROIDS would differ in depth on an asymmetric mesh -> spurious contrast (regresses TPV31, injects a spurious `sigma_n` leak). Evaluating exactly at $\mathbf{x}_f$ is ambiguous for an across-fault `Mode::Coefficient` (`sign(y)` at $y=0$) -> no contrast for TPV6. The eps-offset is the only rule correct for BOTH models.

- `drivers/spatial_dyn_driver.cpp` / the operator: supply the two fault-adjacent elements (`elem_plus`, `elem_minus`) and their transformations + the fault normal `n` and `x_f` for each fault QP. The +/- orientation MUST come from the existing fault-basis `ref_normal` projection (Detailed Requirement 1), not from element index order.
- Set $\eta_p = Z_{p+}Z_{p-}/(Z_{p+}+Z_{p-})$, $\eta_s = Z_{s+}Z_{s-}/(Z_{s+}+Z_{s-})$.

3.2.3 Detailed Requirements

1. The +/- side assignment MUST match the convention `BuildImposedState`/the fault basis use (the `ref_normal=(0,-1,0)` BP5 convention; `+=Q_plus`, `-=Q_minus`). Mis-assigning sides swaps `Zp_plus/Zp_minus` -> wrong reflection sign. Reuse the existing fault +/- determination (`shared_fault_elem1_on_plus` machinery for shared faces; the interior-face `Elem1/Elem2` + normal sign for local faces).
2. Homogeneous-across-fault (TPV31, all TPV/BP5): the two elements have equal material -> `Zp_plus == Zp_minus` -> byte-exact reduction (assert this in a test).
3. (R-101) Shared (cross-rank) fault faces: the local conversion uses only the local element (`wave_operator.inl:3715`), so the PEER side's impedance must come over MPI. The peer rank MUST evaluate ITS fault-adjacent element at the fault-DOF EPS-OFFSET (`x_f - eps*n` on its side, via the same R-102 construction) and send the resulting `(Zp,Zs)` — NOT its centroid/pool material. Sending the peer CENTROID material would reintroduce the R-001 bug on the seam for a depth-/position-varying material (the peer centroid depth != the fault-DOF depth -> spurious across-fault contrast). Add a fault-specific exchange (or a new per-fault-QP field) carrying these eps-offset impedances; do NOT reuse A2's CENTROID neighbour-material exchange for the fault per-side read — A2 (interior-face contrast) legitimately wants centroids, B1 (fault one-sided limit) wants the eps-offset value. They are different quantities. (Benign for TPV6: uniform per side -> centroid == eps-offset. Bites SAFS/CVM and any depth-varying material whose fault is cut by the partition — and the spatial driver does NOT fault-locality-partition, so the fault CAN be cut.)

3.2.4 Edge Cases

- Scalar `WaveOperator` (homogeneous): no per-element pool -> keep the single-material assignment (`Zp_plus==Zp_minus`); the per-side read is matrix-path only.
- A fault DOF whose +/- element straddles a VOLUME material step (fault crosses a layer, e.g., SAFS at `z=-5000`): the eps-offset rule evaluates each side at the fault-QP DEPTH (the probe points share `x_f`'s `z`; they differ only in `y`), so BOTH sides see the layer-consistent material at that depth -> no spurious across-fault contrast from the layer step. (This is the case the centroid approach would get badly wrong.)

3.2.5 Acceptance Criteria

- ☐ Homogeneous bi-material + matrix path: `Zp_plus == Zp_minus` for every fault DOF (byte-exact vs today) — new assert test.
- ☐ (R-001 regression guard) TPV31 `depth_profile_1d` on an ASYMMETRIC mesh, including a fault DOF adjacent to the `z=-5000` layer step: `Zp_plus == Zp_minus` to rel 1e-12 for EVERY fault DOF (the eps-offset rule must NOT manufacture a contrast from mesh asymmetry or the layer step). This test MUST FAIL if the implementation uses element centroids.
- ☐ TPV6 fixture (two half-spaces): fault DOFs get `Zp_plus = Zp(near)=6000*2670`, `Zp_minus = Zp(far)=3750*2225` (mapped per the `ref_normal` orientation, R-006), `eta_p` the harmonic mean — unit test on a small two-material fault mesh.
- ☐ (R-101) `np=2` with a DEPTH-VARYING material and the fault CUT by the partition: the shared-fault `Zp_minus/Zs_minus` (peer side, over MPI) equal the serial eps-offset values to rel 1e-12 — i.e. the peer sends its eps-offset impedance, not its centroid, so no spurious across-fault contrast appears on the seam.
- ☐ `np=2` homogeneous-across-fault: shared-fault `Zp_plus == Zp_minus` (byte-exact).

3.2.6 Dependencies: depends on B0; required by B2, C2.

3.3 Phase B2: relax the homogeneity guards for the matrix path

3.3.1 Goal

`Zp_plus != Zp_minus` runs on the `BimaterialWaveOperator` (per-side A) instead of aborting; the scalar `WaveOperator` still aborts (cannot apply per-side A).

3.3.2 Files to Modify

- `dynamic/fault_face_flux.cpp` (:359,499,778,912,1030): in EACH guarded variant, replace the unconditional `MFEM_VERIFY(homog_ok(...))` with a flag the operator sets: `FaultFaceFlux::SetPerSideFluxApplied(bool)` (default false). When true, SKIP the homogeneity abort (the caller guarantees per-side A at the conversion site). When false, keep the abort (scalar path / unverified callers).
- `dynamic/bimaterial_wave_operator.inl` (ctor or `SetFaultFlux`): call `fault_flux_->SetPerSideFluxApplied(true)` — ONLY the matrix operator, which applies per-side A via `FluxForElem_`.
- Scalar `WaveOperator`: never sets the flag -> abort preserved.

3.3.3 Detailed Requirements

1. The flag is the single switch; do NOT delete the guards (defense for the scalar path and for any future caller that forgets per-side A).
2. (R-005) Relax the guard ONLY in variants the B0 TABLE marks per-side-A = YES AND that are used by a matrix-path driver. If the B0 table shows ANY used variant whose conversion site is NOT per-side A, do NOT relax that variant via the flag — keep its abort and open a follow-up to fix that conversion site first. Prefer a per-variant guard over the single flag if the table is mixed.
3. The relaxation is valid ONLY because B0 verified per-side A at every relaxed variant’s conversion site; reference the B0 verdict doc + the table row in each code comment.

3.3.4 Edge Cases

- A bi-material fault DOF reaching the scalar path (misconfiguration) -> abort with the existing clear message (now “...and SetPerSideFluxApplied was not set”).

3.3.5 Acceptance Criteria

- ☐ Matrix path + `Zp_plus != Zp_minus`: runs (no abort); produces finite flux matching the B0 analytic Riemann to 1e-9 on a single-face fixture.
- ☐ The relaxation enables the bi-material fault for BOTH interior-flux choices, since the fault-flux guard is independent of the interior flux: matrix + `mixed_flux=none` + per-side fault runs under **ADER** (TPV6 Arm 1), AND matrix + `mixed_flux=adjacent` + per-side fault runs under **RK** (TPV6 Arm 2). Both exercised via `seas_tpv6_smoke` in C2.
- ☐ Scalar path + `Zp_plus != Zp_minus`: still aborts.
- ☐ Homogeneous matrix path: byte-exact (flag set but `Zp_plus==Zp_minus` -> same flux).

3.3.6 Dependencies: depends on B0, B1; required by C2.

3.4 Phase B3: across-fault material schema

3.4.1 Goal

Express “material A on one side of the fault, material B on the other” (TPV6/7) and, generally, any material field that differs across the fault, in the spatial config.

3.4.2 Files to Modify

- `dynamic/heterogeneous_material.hpp` + `spatial/code/spatial_friction.{hpp,cpp}`: add a material kind `halfspace_across_fault` with two `{vp,vs,rho}` triples and a plane (point + normal, default the

fault plane) selecting side by the sign of $(x - x0).normal$. `MakeHalfspaceMaterial(...)` returns a `MaterialField (Mode::Coefficient)` the `BimaterialWaveOperator` consumes via the per-element pool.

- TOML schema: `[material] kind="halfspace_across_fault" + [material.halfspace.side_far]/[material.halfspace.side_near]` + optional `plane_point_m/plane_normal` (default = fault).
- Schema doc: `safs/project_7.0_alternative/document/spatial_friction_config_schema.md`.

3.4.3 Detailed Requirements

1. (R-006) The half-space split MUST agree with the fault +/- convention used in B1. Define `side_near := (x - x0).n > 0 -> Q_plus` and `side_far := (x - x0).n < 0 -> Q_minus`, consistent with the fault `ref_normal=(0,-1,0)` convention. The value exactly on the plane ($(x-x0).n == 0$) is NEVER queried because B1's eps-offset rule evaluates at $x_f +/- eps*n$ (never at x_f); document this dependency. Assert the `side_near -> Q_plus` mapping in a parse test on a 2-element fixture straddling the plane, and document it in the TPV6 config header so the fast side (near, `vp=6000`) and slow side (far) land on the intended y-sides.
2. Reuse the existing `MaterialField/per-element-pool` machinery (no new operator code); this kind just produces per-element materials.
3. Composability: a future SAFS run can use the CVM sidecar (volume) AND a fault-side override — out of scope here, but the schema must not preclude it (keep the kind orthogonal to `[velocity]`).

3.4.4 Acceptance Criteria

- ☐ `test_spatial_friction_config / test_tpv_config_parse` extended: the TPV6 triples parse; side selection by sign is correct on a 2-element fixture straddling the plane.
- ☐ `MakeHalfspaceMaterial` yields per-element materials that, fed to `BimaterialWaveOperator`, give the expected `Zp_plus/Zp_minus` at the fault (ties B1 + B3).

3.4.5 Dependencies: depends on B1; required by C2.

4 PART C — Problem setups

4.1 Phase C1: TPV31 (bi-material VOLUME) — contrast-guard fix jobs ONLY

TPV31's fault is material-SYMMETRIC across y (depth-only profile), so TPV31 needs ONLY the bi-material VOLUME fix (Part A) — it does NOT exercise the per-side fault Riemann (Part B). Scope here is exactly: ship the contrast-guard-enabled TPV31 jobs.

4.1.1 Files to Create

- `jobs/tpv31_spatial/mixedflux_rk45/tpv31_p1_rk45_mixedflux_50m_normal_contrastguard.sbatch` — copy of the gold p1 job (`mixedflux_p1_rk45`) + `--mixed-flux-contrast-tol 0.05`.
- `jobs/tpv31_spatial/mixedflux_rk45/tpv31_p2_rk45_mixedflux_50m_normal_contrastguard.sbatch` — the p2 variant (the run that showed the catastrophic interface leak this session). Both keep matrix + mixed-flux adjacent + RK45 + the normal (asymmetric) 50 m mesh; ONLY the new `--mixed-flux-contrast-tol 0.05` flag is added.
- (Optional control) the existing upwind/symmetric and pure-upwind jobs are retained unchanged as the leak-free reference; no new physics.

4.1.2 Detailed Requirements

1. (R-103) TPV31 needs only Part A. On a Part-A-only tree the fault setup is unchanged (single-material both sides). IF this is built on the unified branch (Part B present), B1's eps-offset read MUST yield `Zp_plus == Zp_minus` for every TPV31 fault DOF (the B1 R-001 regression test enforces this), so TPV31 stays on the homogeneous-fault path either way.

2. The contrast guard reclassifies ONLY the corridor faces crossing the 2400/5000/10000 m layer steps; the startup histogram must show a nonzero **reclassified** N/M for TPV31 (proves the guard engaged) — if it shows 0, the flag was not applied (R-006 ordering).

4.1.3 Acceptance Criteria

- ☐ Both sbatch parse-check / --dry-run --verify-dispatch show mixed flux contrast tol: 0.05 and a nonzero reclassified count.
- ☐ Frontera (user-submitted): sigma_n at dp024/dp050/dp100 drops from the 16/9/64 MPa collapse toward the mid-layer floor; V_strike/slip within ~5% of the guard-off baseline (rupture not damped).
- ☐ Guard OFF default: byte-identical to the current gold runs.

4.1.4 Dependencies: depends on Part A (A1-A4). Does NOT require Part B; if Part B is present it must remain byte-exact for TPV31 (B1 R-001 test).

4.2 Phase C2: TPV6 / TPV7 (bi-material FAULT) — NEW problem

4.2.1 Spec (from 2007RuthRalphletter2.pdf, MKS)

- Fault: vertical right-lateral strike-slip, reaches the FREE SURFACE; rupture box 30000 m (strike) x 15000 m (depth); nucleation 3000x3000 m centred at strike=15000, depth=7500.
- Material (homogeneous within each side, jump across the fault):
 - TPV6 (well-posed, high contrast): far side vp1=3750, vs1=2165, rho1=2225; near side vp2=6000, vs2=3464, rho2=2670.
 - TPV7 (ill-posed, lower contrast): far side vp1=5000, vs1=2887, rho1=2670; near side same as TPV6 near side. (Same config except the far-side triple.)
- Friction LSW: mu_s=0.677, mu_d=0.525, d_c=0.40 m, sigma_n=120 MPa. tau_strike(t=0)=70 MPa outside patch, 81.6 MPa inside; tau_dip(t=0)=0. Barriers (box edges + bottom): mu_s=10000. Nucleation = TPV3-style overstress (the in-patch tau exceeds static yield 81.24 MPa at t=0; instantaneous, like TPV3).
- Output: stations report displacement & velocity on EACH side (not slip); contours use slip-rate. 100 m element size.

4.2.2 Run arms (REQUIRED — both must be supported and tested)

TPV6's bi-material contrast is AT the fault (per-side fault Riemann, Part B); each half-space is uniform, so EVERY fault-adjacent corridor face is within one material → the volume contrast guard (Part A) is INERT for TPV6 (no corridor crosses a contrast). Two production schemes:

- **Arm 1 — bi-material UPWIND on a SYMMETRIC mesh, ADER.** interior_flux="matrix", mixed_flux="none", --time-integrator ADER (e.g. ADER-O2 at p1). Pure upwind everywhere + per-side bi-material fault Riemann (Part B). The symmetric near-fault mesh removes the +y/-y dissipation-asymmetry seed. This validates that **matrix + upwind + per-side fault runs under ADER** (the existing ADER guard only blocks matrix+mixed, not matrix+upwind — so this arm needs ONLY Part B, not Part A).
- **Arm 2 — MIXED flux on the (asymmetric) mesh, RK.** interior_flux="matrix", mixed_flux="adjacent", --time-integrator rk45. Central on the (uniform-per-side) corridor + upwind bulk + per-side fault Riemann. Central is non-dissipative → ADER-unstable → RK required (existing guard). Beneficial on an asymmetric mesh because central removes the near-fault upwind-dissipation asymmetry while the per-side fault Riemann carries the (physical) bi-material coupling. The contrast guard is present but reclassifies 0 faces (assert this: TPV6 histogram shows **reclassified** 0/M).

Both arms use the SAME per-side fault Riemann (Part B) and the SAME **halfspace_across_fault** material (B3). sigma_n is PHYSICALLY non-constant in BOTH (do not apply a flatness criterion; compare to SCEC).

4.2.3 Files to Create

- tpv6/configs/tpv6.toml (and tpv7.toml): canonical-frame config — [material] kind="halfspace_across_fault" with the two triples (B3); [friction.slip_weakening] mu_s/mu_d/d_c; barriers via spatial regions

($\mu_s=10000$ outside the 30x15 km box); free-surface BC (top); `interior_flux="matrix"`. Apply the canonical rotation TPV uses (fault $y=0$, strike $+x$, depth $-z$) consistent with TPV31. Keep `mixed_flux/integrator` as CLI-overrideable so ONE config drives BOTH arms (the sbatch pins the scheme); default the config to Arm 1 (`mixed_flux="none"`).

- Meshes (Gmsh v2.2, 100 m): `tpv6/mesh/tpv6_100m_symmetric.geo` (y-mirror-symmetric near-fault, for Arm 1) and `tpv6/mesh/tpv6_100m.geo` (standard/asymmetric, for Arm 2).
- Sbatch jobs:
 - `tpv6/jobs/upwind_ader/tpv6_p1_ader02_upwind_100m_symmetric.sbatch` — Arm 1: `interior_flux=matrix` (config) + `--mixed-flux none --ader-order 2` on the symmetric mesh. (Also a `tpv7_...` twin.)
 - `tpv6/jobs/mixedflux_rk45/tpv6_p1_rk45_mixedflux_100m.sbatch` — Arm 2: `--mixed-flux adjacent --time-integrator rk45` on the standard mesh. (Plus `tpv7_.`)
- [nucleation] (R-003): add and use a NEW kind `instantaneous overstress_square` — UNIFORM `delta_tau` over the square patch $\{|x-x_0| \leq L/2 \text{ AND } |z-z_0| \leq L/2\}$, $L=3000$ m, centred at strike=15000/depth=7500, NO taper, applied to the strike component at $t=0$ so in-patch `tau_strike = 81.6` MPa ($= 70 + 11.6$). Do NOT reuse `instantaneous overstress_circular` (its cosine-tapered circular patch does not match the TPV3/TPV6 spec — wrong shape and wrong in-patch profile). Implement the kind in `dynamic/spatial_nucleation.hpp` mirroring the circular one but with the square, untapered support.
- `tpv6/mesh/tpv6_100m.geo` — 100 m mesh, fault $y=0$, free surface at $z=0$, box per spec, Gmsh v2.2. The two material half-spaces are by element centroid sign of y (the B3 plane) — the mesh need not pre-tag materials (the material field assigns per element).
- `dynamic/tpv6_stations.hpp` (NEW) — per-SIDE station writer, mirroring `tpv31_stations.hpp` but emitting the bi-material two-sided columns. (R-004) VELOCITY is read from Q directly per side; DISPLACEMENT is NOT a state variable (the operator evolves velocity-stress) — maintain a per-station, per-side displacement ACCUMULATOR $d += v*dt$ (trapezoidal over each macro step), initialized to 0 at $t=0$, written at the station output cadence. Document the integration scheme in the writer header.
- `tpv6/visualize_results.py` — overlay vs SCEC TPV6/7 reference (two-sided displacement/velocity; slip-rate contours).
- `tpv6/benchmark_data/` — placeholder for SCEC reference traces (user supplies).

4.2.4 Detailed Requirements

1. The $\pm$ side material mapping (B1/B3) MUST match the spec’s “near”/“far”: verify the sign convention so the FAST side (near, $vp_2=6000$) and SLOW side (far) land on the intended y -sides; document in the config header.
2. Free surface at $z=0$ with the bi-material: confirm the free-surface flux (`FreeSurfaceTotal`) composes with per-side A near the fault-surface intersection (edge case; flag for B0/verification).
3. `sigma_n` is PHYSICALLY non-constant for TPV6 (bi-material coupling) — the station output and any drift diagnostics must NOT treat `sigma_n` change as a leak (opposite of TPV31). Document prominently.
4. (R-104a) COORDINATE FRAME: define the canonical origin in the `tpv6.toml` header, matching TPV31’s convention. If the fault is centered at $x=0$ (as TPV31), the nucleation patch center is $x_0=0$ (NOT the spec’s 15000) and $z_0=-7500$; the spec’s strike=15000 is the spec-frame value. The R-003 square-patch test MUST use the CONFIG’s frame, not the spec frame, so the patch lands on the fault center.
5. (R-104b) STATION VELOCITY: the per-side velocity is the BULK particle velocity of the $\pm$ element trace at the station (`Q[VX..VZ]` of each side), i.e. the ground motion — NOT the imposed-state (fault-Riemann) velocity.
6. (R-104c) RESTART: the per-station per-side displacement accumulator (R-004) is integrated state — include it in the checkpoint and restore it on resume (zero-init ONLY on a fresh start), so resumed displacement is continuous. (TPV6 is short and likely single-segment, but state it so a restart does not silently zero displacement.)

4.2.5 Edge Cases

- Fault-surface corner (free surface meets bi-material fault): per-side A + free-surface flux interaction — verify no spurious normal-stress generation beyond the physical bi-material effect.
- TPV7 ill-posed regime: the bi-material problem is grid-dependent by design; document that exact convergence

is not expected (match the SCEC ensemble spread, not a single reference).

4.2.6 Acceptance Criteria

- `tpv6.toml/tpv7.toml` parse and `--dry-run --verify-dispatch` resolves matrix + per-side fault on the 100 m mesh (banner shows `Zp_plus != Zp_minus`).
- **Arm 1 (matrix + UPWIND + per-side fault under ADER):** `seas_tpv6_smoke` with `--mixed-flux none --ader-order 2` on the symmetric mesh runs WITHOUT abort (proves the existing ADER guard does not block matrix+upwind+bimaterial-fault) and produces bounded, NaN-free two-sided station output.
- **Arm 2 (matrix + MIXED + per-side fault under RK):** `seas_tpv6_smoke` with `--mixed-flux adjacent --time-integrator rk45` runs without abort; the startup histogram reports `reclassified 0/M` (the contrast guard is correctly INERT for TPV6 — no corridor crosses a contrast).
- A matrix + MIXED + per-side fault run under **ADER** still ABORTS (the central+ADER guard is unaffected by the fault-flux per-side relaxation).
- (R-007 free-surface/bi-material corner) With the fault LOCKED (strength above `|tau_trial|`) and a quiescent initial state, on-fault `sigma_n` at the shallowest DOFs (`z -> 0`) shows NO drift over `N` steps (no slip -> zero bi-material coupling), so the free-surface flux + per-side A composition introduces no spurious normal stress at the surface-fault line.
- Frontera (user-submitted): TPV6 vs SCEC reference — rupture time, two-sided velocity, and the bi-material `sigma_n` signature within the SCEC ensemble spread.

4.2.7 Dependencies: depends on B1, B2, B3.

5 PART D — Testing Strategy

- Unit (local):
 - A1 predicate; A4 two-sided dissipation mechanism; A2 face-set (serial + np=2).
 - B0 `seas_test_bimaterial_fault_riemann` (analytic R/T coefficients).
 - B1 per-side impedance assignment (homogeneous byte-exact; TPV6 two-half-space).
 - B2 guard relaxation (matrix runs, scalar aborts, homogeneous byte-exact).
 - B3 schema parse + side selection.
 - C2 `seas_tpv6_smoke`.
- Parallel (local): np=2 shared fault face per-side impedance + contrast-guard shared face consistency.
- Serial planar harness: keep the TPV31 volume-leak gate; optionally extend with a bi-material-fault variant.
- Production (Frontera, user-submitted): TPV31 volume-guard A/B; TPV6/7 vs SCEC.
- Cross-checks: analytic 1D bi-material Riemann (B0); `drdg3d` / SeisSol bi-material flux (`/Users/chunhuizhao/projects/dro` the v9.0.0 comparison doc); SCEC TPV6/7 ensemble.

6 Risk Assessment

- R-A (the prior review’s R-001..R-007) — carried into A1/A2/A4 above.
- R-B0 (CRITICAL gate): if B0 finds the per-side imposed-state Riemann is NOT correct for unequal Z (a hidden homogeneous assumption beyond the call site), B2 is blocked and a real Riemann fix is needed. Mitigation: B0 is an explicit gate with the analytic R/T test; do not relax guards over a defect.
- R-B1 (HIGH): +/- side mis-assignment swaps `Zp_plus/Zp_minus -> wrong reflection sign`, silently plausible output. Mitigation: reuse the EXISTING fault +/- convention; assert homogeneous reduction (`Zp_plus==Zp_minus`) byte-exact; analytic R/T sign test in B0.
- R-B2 (MEDIUM): relaxing the guard could let a scalar-path bi-material fault through if the flag is mis-set. Mitigation: flag set ONLY in the matrix operator; scalar abort preserved; explicit test “scalar + bimaterial -> abort”.
- R-MPI (MEDIUM): shared fault face per-side material or +/- orientation disagrees across ranks -> non-conservative / wrong flux. Mitigation: reuse `shared_fault_elem1_on_plus_`; np=2 tests for both impedance and orientation.

- R-C2-freesurface (MEDIUM): free surface x bi-material fault corner (TPV6 reaches the surface) — untested interaction. Mitigation: B0 edge-case + a smoke test; flag for a dedicated check if the corner shows spurious normal stress.
- R-C2-illposed (LOW/expected): TPV7 is grid-dependent by design — validate against the SCEC ensemble spread, not a single trace.
- R-scope (MEDIUM): this plan is large. Order: PART A (volume guard, self-contained, highest immediate value for TPV31) first; PART B (B0 verify -> B1 -> B2) next; PART C2 (TPV6) last. Each phase is independently testable and leaves the tree green.